



Hardware e Software per Grafica Interattiva





Sommario

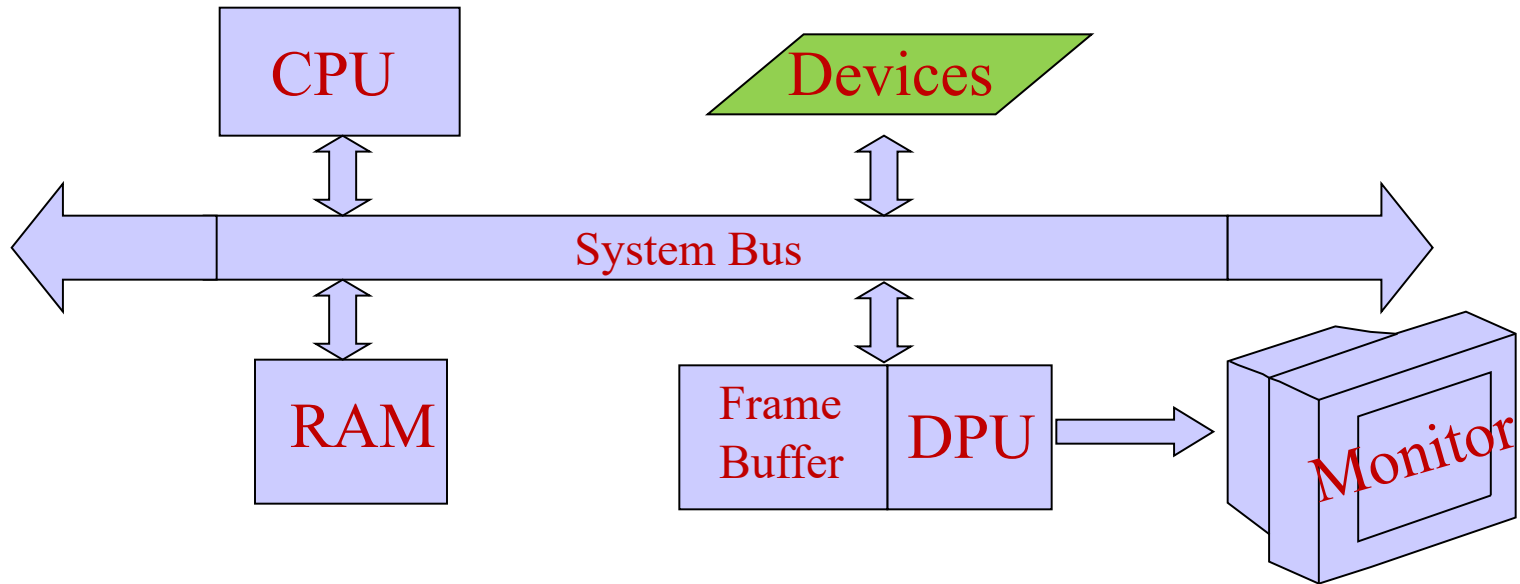
Architetture Grafiche (dagli anni '80 ad oggi)

- Display, Frame Buffer, DPU
- Sistemi Grafici
- Pipeline Grafica (transform & rendering)
- Schede grafiche e GPU

Dispositivi Grafici Interattivi

- Mouse, ...
- Il problema dell'interattività
- Coda degli Eventi
- Programmazione Event-Driven

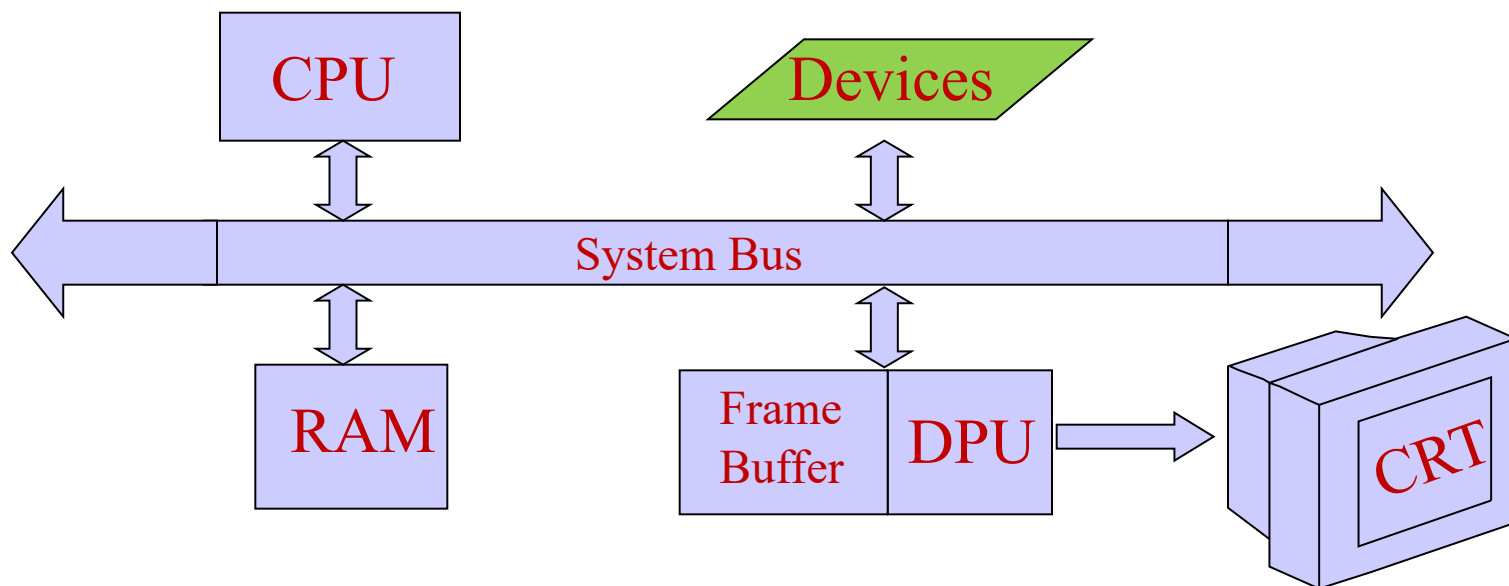
Architettura Grafica (anni '80)



- Un disegno (o immagine) su uno schermo grafico è generato punto per punto (o pixel per pixel)
- disegnare consiste nel "settare" l'intensità luminosa di ogni pixel

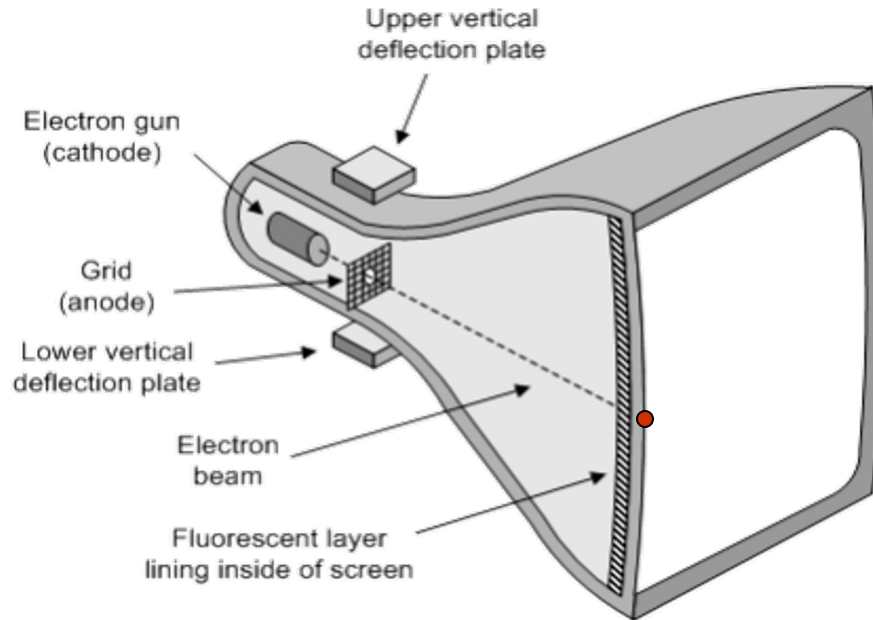
pixel = picture element

Componenti Hardware



- **CRT**: Monitor/Display su cui visualizzare l'immagine
- **Frame Buffer**: Memoria RAM in cui si memorizza l'intensità o colore di ogni pixel
- **DPU**: Processore grafico che scandisce il contenuto del Frame Buffer e produce un segnale per il Monitor

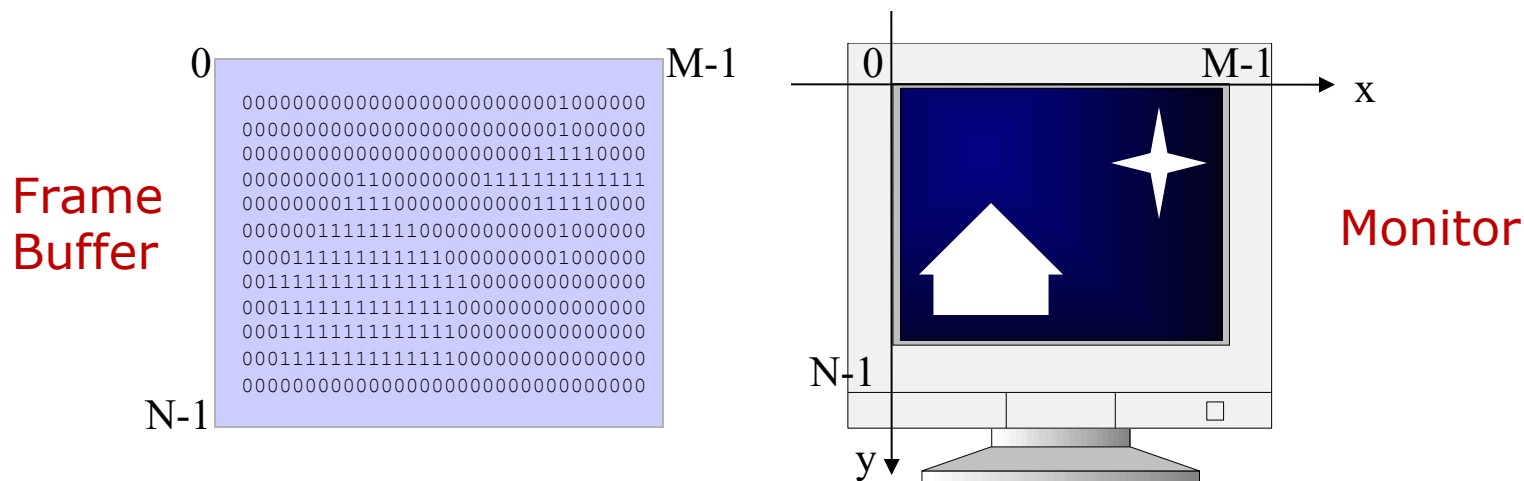
Display: Cathod Ray Tube(CRT)



- Il voltaggio applicato al “cannone di elettroni” determina il numero di elettroni emessi
- L'intensità di un punto illuminato dipende dal numero di elettroni del pennello
- La luce emessa dal fosforo dura solo una piccola frazione di secondo
- Si deve rinfrescare l'immagine circa 60 volte al secondo per evitare l'effetto tremolio (FLICKERING); CRT a rinfreso.

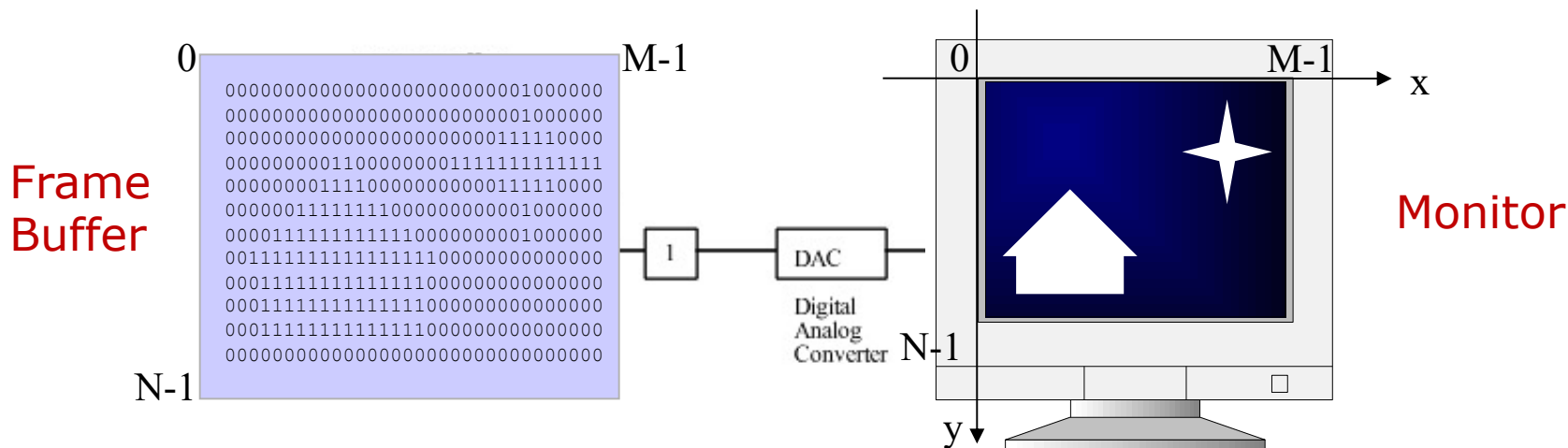
Frame Buffer

Memoria RAM identificata con un array bidimensionale



- Il Frame Buffer più piccolo: **1 bit per pixel**
- Un "1" nel Frame Buffer corrisponde ad un pixel acceso nel Monitor (sistema monocromatico)
- Una riga nel Frame Buffer corrisponde ad una scanline nel Monitor
- Un elemento in una particolare riga e colonna contiene l'informazione relativa all'intensità (o colore) della corrispondente posizione **[x,y]** sullo schermo

Frame Buffer

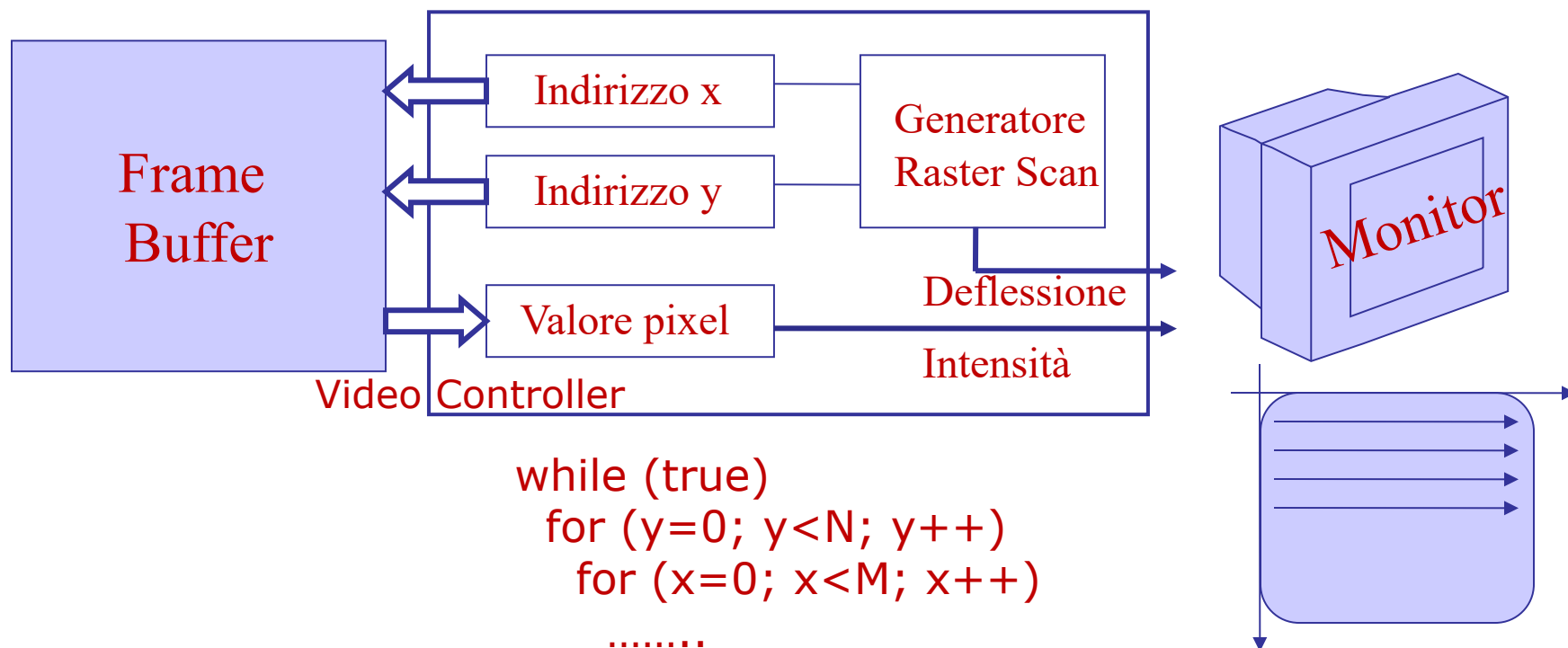


- Un sistema 512x512 richiede $512 \times 512 = 2^{18} = 262.144$ bit di memoria
- Ma il Frame Buffer è un array monodimensionale, per cui dato $[x, y]$, la relativa posizione di memoria viene calcolata da:

$$st_fm + M * y + x$$

- Poiché un Frame Buffer è un dispositivo **digitale**, mentre il CRT è **analogico**, deve essere effettuata una conversione digitale-analogica usando un DAC (Digital Analog Converter)

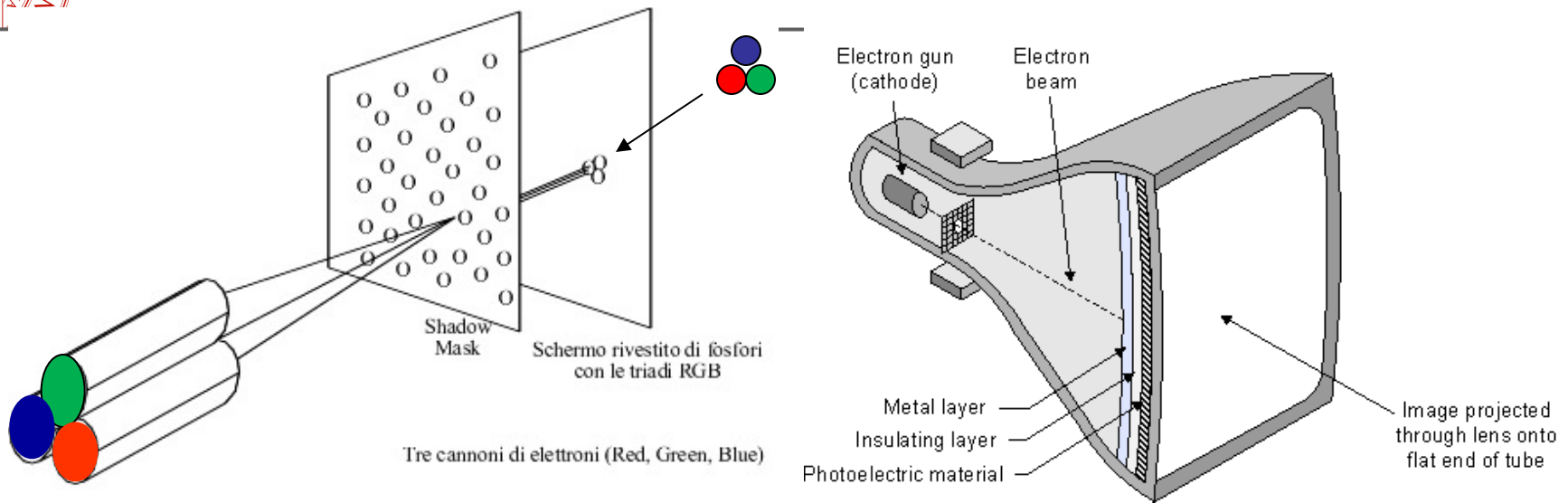
DPU (Display Processor Unit)



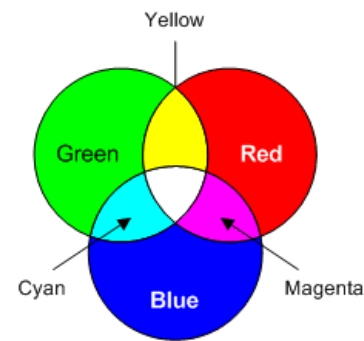
- A sistema funzionante, per accendere un pixel nella posizione $[x,y]$, sarà sufficiente inserire nella corrispondente posizione del Frame Buffer, l'informazione "1" (acceso in un sistema monocromatico).
- Tutti i software grafici possiedono una funzione per far questo:

`draw_point(int x, int y, unsigned long color)`

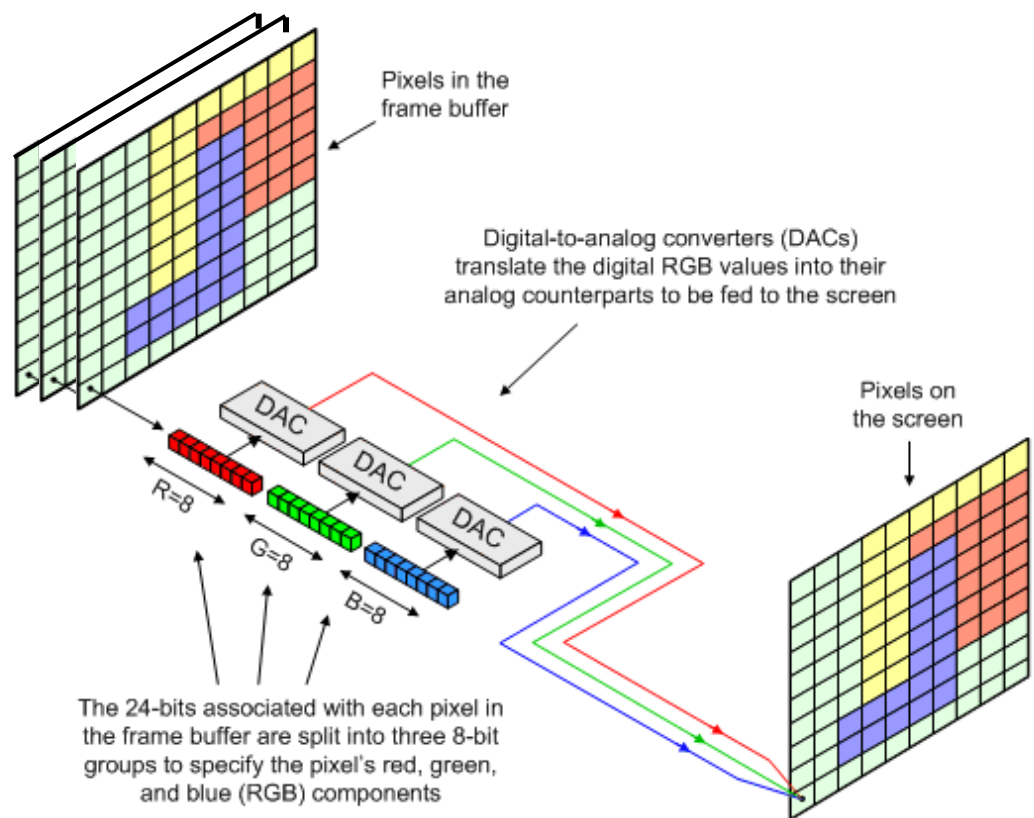
CRT a Colori (Shadow Mask)



- La Shadow Mask consiste in una sottilissima lastra metallica con piccolissimi fori, montata vicino allo strato di fosforo;
- La Shadow Mask è perfettamente posizionata, così che i tre pennelli di elettroni (uno per il **rosso**, uno per il **verde** ed uno per il **blu**), possano colpire solo un tipo di fosforo;
- In figura è presentato un Delta CRT in quanto i punti di fosforo sono disposti a triangolo (Δ).



Frame Buffer 24 bit/color



- Oggi il più comune schema di Frame Buffer è ad **8 piani per colore**, per un totale di **24 piani**. Ogni gruppo di piani guida un DAC (Digital-Analogic Converter) ad 8 bit; questi sono combinati in $(2^8)^3 = 2^{24} = 16.777.216$ possibili colori (Frame Buffer Full Color)

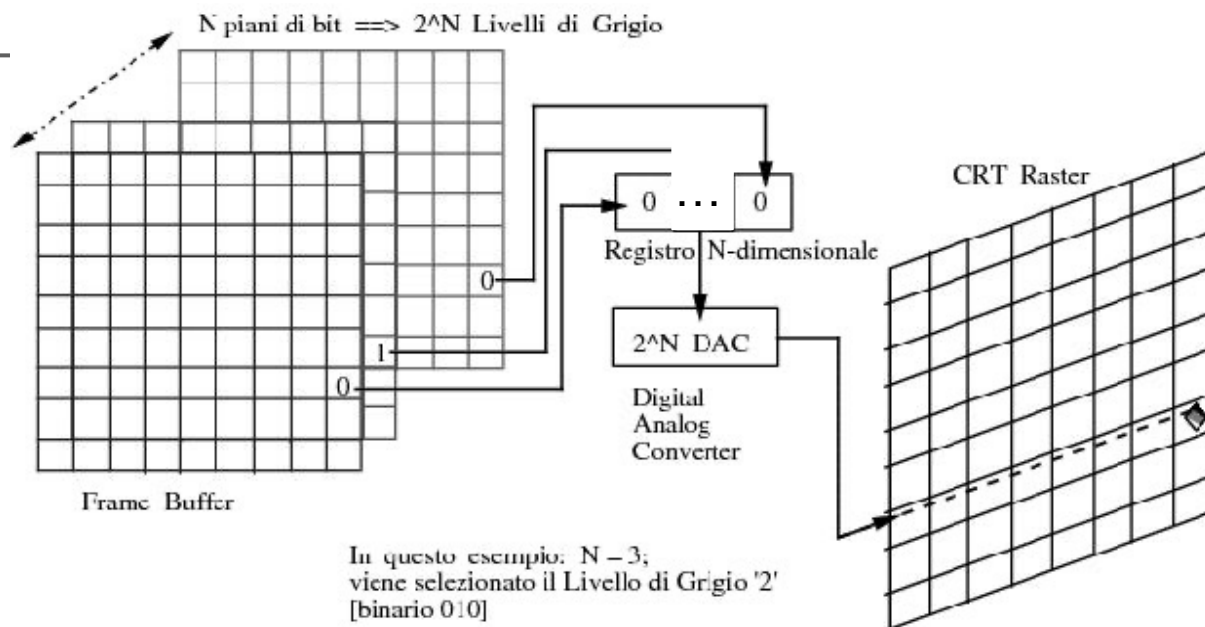
Monitor e Display negli anni

Il componente principale di un monitor è il display, cioè il dispositivo elettronico per la visualizzazione. In base alla tecnologia usata si hanno le seguenti tipologie di display:



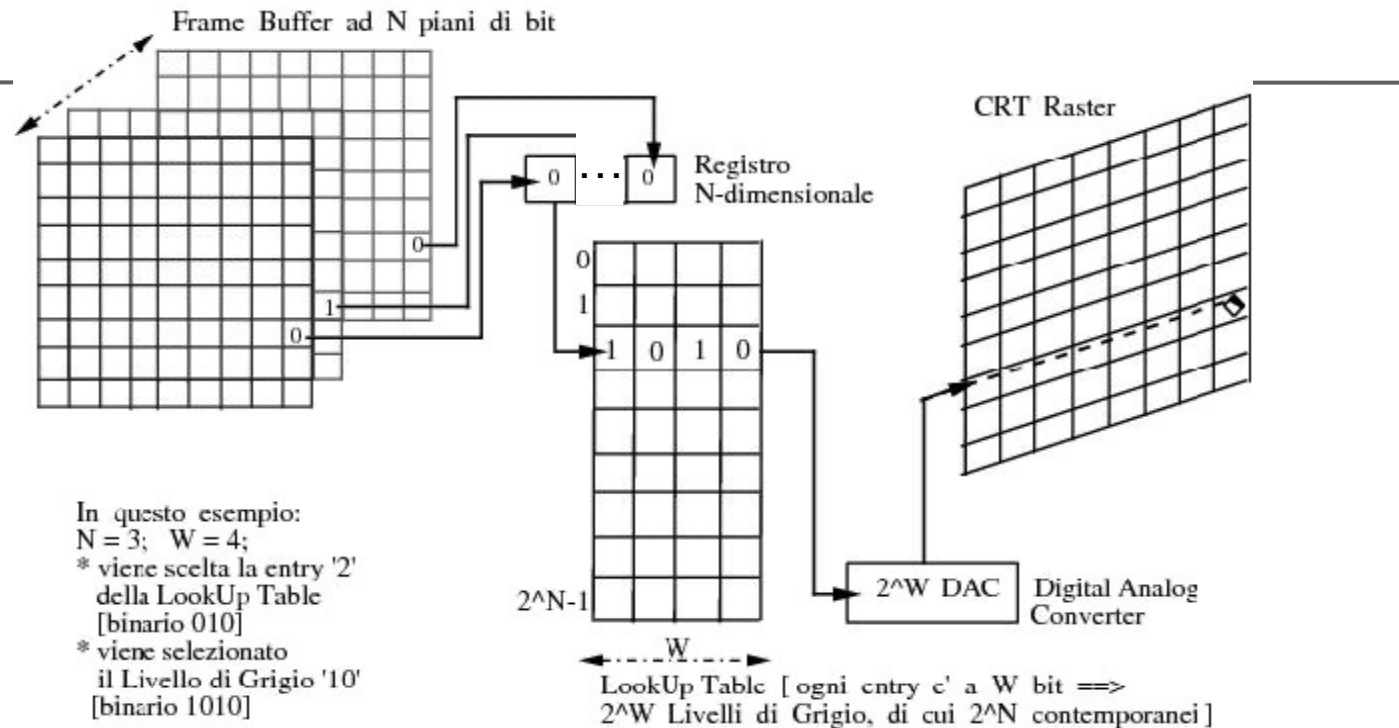
- CRT (Cathod Ray Tube)
- LCD (Liquid Crystal Display)
- TFT (Thin Film Transistor)
- LED (OLED acronimo di Organic Light Emitting Diode)
- PDP (Plasma Display Panel)
- 3D

Tipi di Frame Buffer negli anni



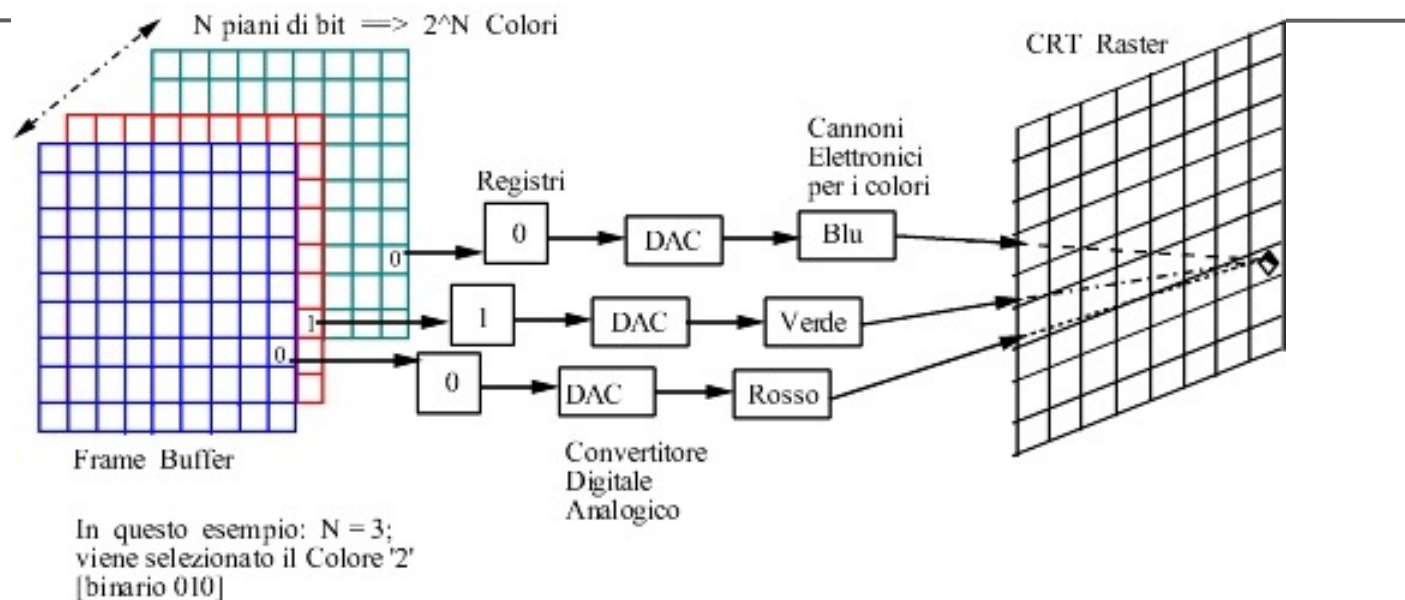
- Per più livelli di grigio è necessario un Frame Buffer con più di 1 piano;
- L'intensità di ciascun pixel sul CRT è controllata dalla corrispondente locazione del pixel in ciascuno degli N piani;
- Il valore binario (0 o 1) di ciascuno degli N piani viene caricato nella corrispondente posizione di un registro;
- Il risultante numero binario viene interpretato come un livello di intensità fra 0 (scuro) e $2^N - 1$ (massima intensità).
- Questo viene convertito in un voltaggio analogico fra 0 ed il max
- Sono possibili 2^N livelli (3 piani, $512 \times 512 = 786.432$ bit di memoria).

Tipi di Frame Buffer negli anni



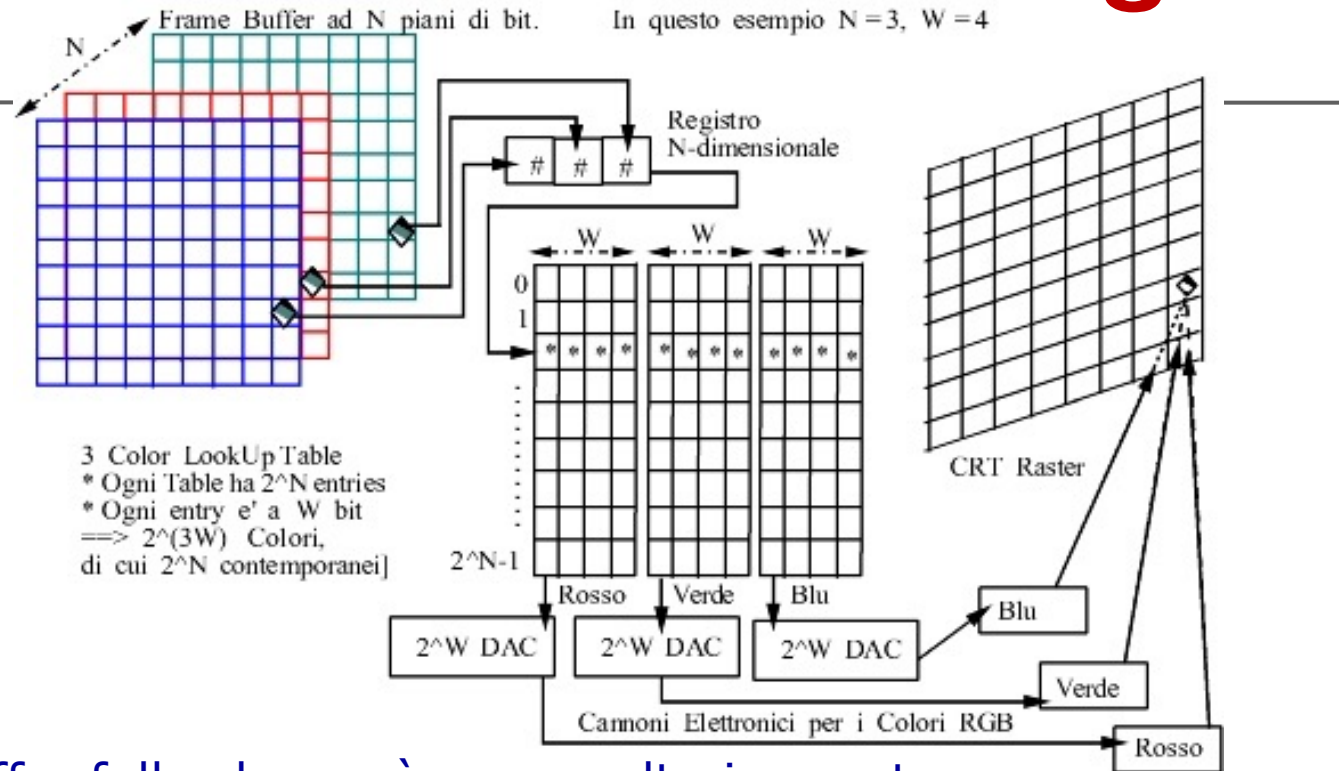
- Mediante l'uso di una **lookup table** si può ottenere un aumento del numero di livelli di intensità al prezzo di un modesto aumento di memoria.
- Il numero risultante dei bit dei piani nel Frame Buffer viene usato come indice nella lookup table.
- La lookup table deve contenere **2^N entry**; ogni entry nella lookup table è formato da **W bit**. W può essere maggiore di N; allora si dice che sono disponibili 2^W intensità, ma solo 2^N di queste sono contemporaneamente utilizzabili.

Tipi di Frame Buffer negli anni



- Un semplice Frame Buffer per il colore si può ottenere con 3 piani ognuno per uno dei colori primari;
- I tre colori primari, **RED**, **GREEN** e **BLUE**, sono combinati nel CRT per avere gli otto colori (Black, Red, Green, Blue, Yellow, Cyan, Magenta, White).
- Possono essere utilizzati dei piani aggiuntivi per ciascuno dei tre colori; come detto, il più comune ed oggi standard è uno schema di Frame Buffer con **8 piani per colore**.

Tipi di Frame Buffer negli anni



- Il Frame Buffer full color, può essere ulteriormente espanso con l'utilizzo dei piani come indici di **color lookup table**;
- Per N piccolo (≤ 4) risulta più vantaggioso se le lookup table vengono implementate contigue con 2^N entry (vedi esempio)
- Per **N piani/colore** per ogni color lookup table di W bit, si possono avere contemporaneamente $(2^3)^N$ colori da una palette di $(2^3)^W$ possibili colori; per esempio per $N=8$ e $W=10$ si possono ottenere 2^{24} colori da una palette di 2^{30} .



Storia delle schede grafiche

CPU: computer di uso generale ('60);

VGA (Video Graphic Array) Hardware controller (**DPU**) (anni '80):
sistema grafico speciale;

Graphics Hardware Unit ('90): sistema grafico a pipeline (circuiti VLSI (Very Large Scale Integration)), Silicon Graphics International (SGI) e Evans & Sutherland progettano un costoso multichip;

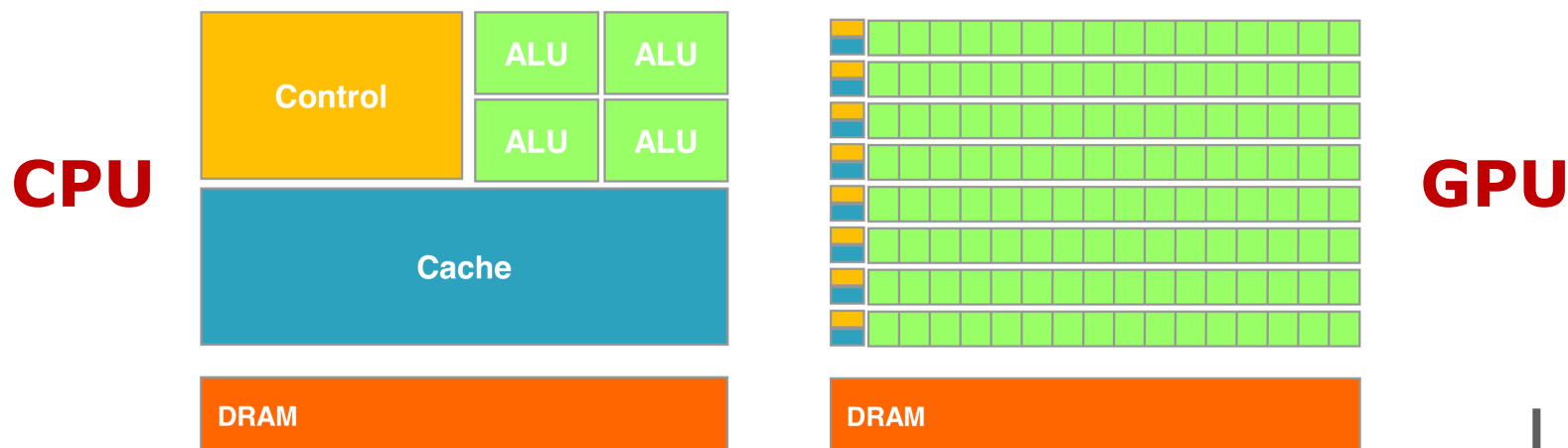
GPU (Graphics Processor Unit) (fine '90): GPU a chip singolo, più economico, in PC e console per videogiochi;

GPGPU (General Purpose GPU) (inizio anni 2000, la svolta nel 2006-2007 con NVIDIA CUDA).

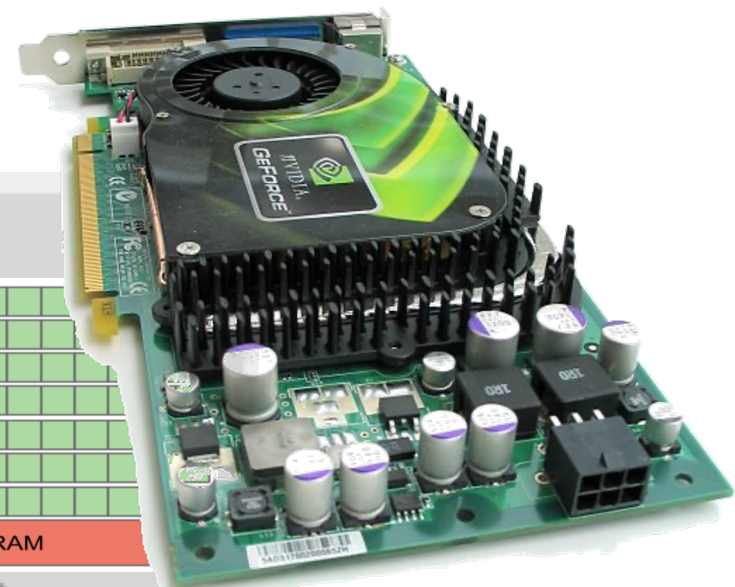
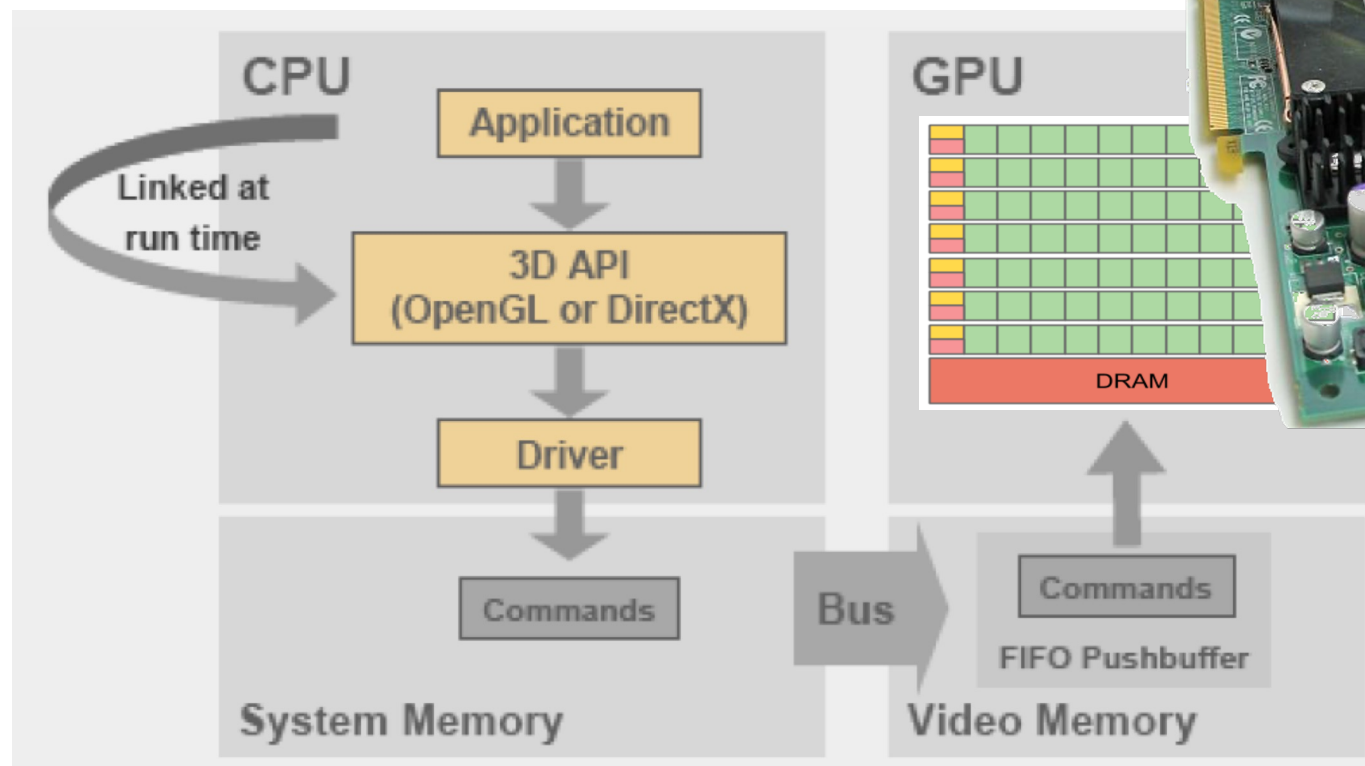
GPU vs CPU

Le **GPU** sono processori specializzati per problemi classificati come **intense data-parallel computations**, cioè la stessa computazione/algorithmo viene eseguita/o su molti dati differenti in parallelo (modello di computazione **SIMD**)

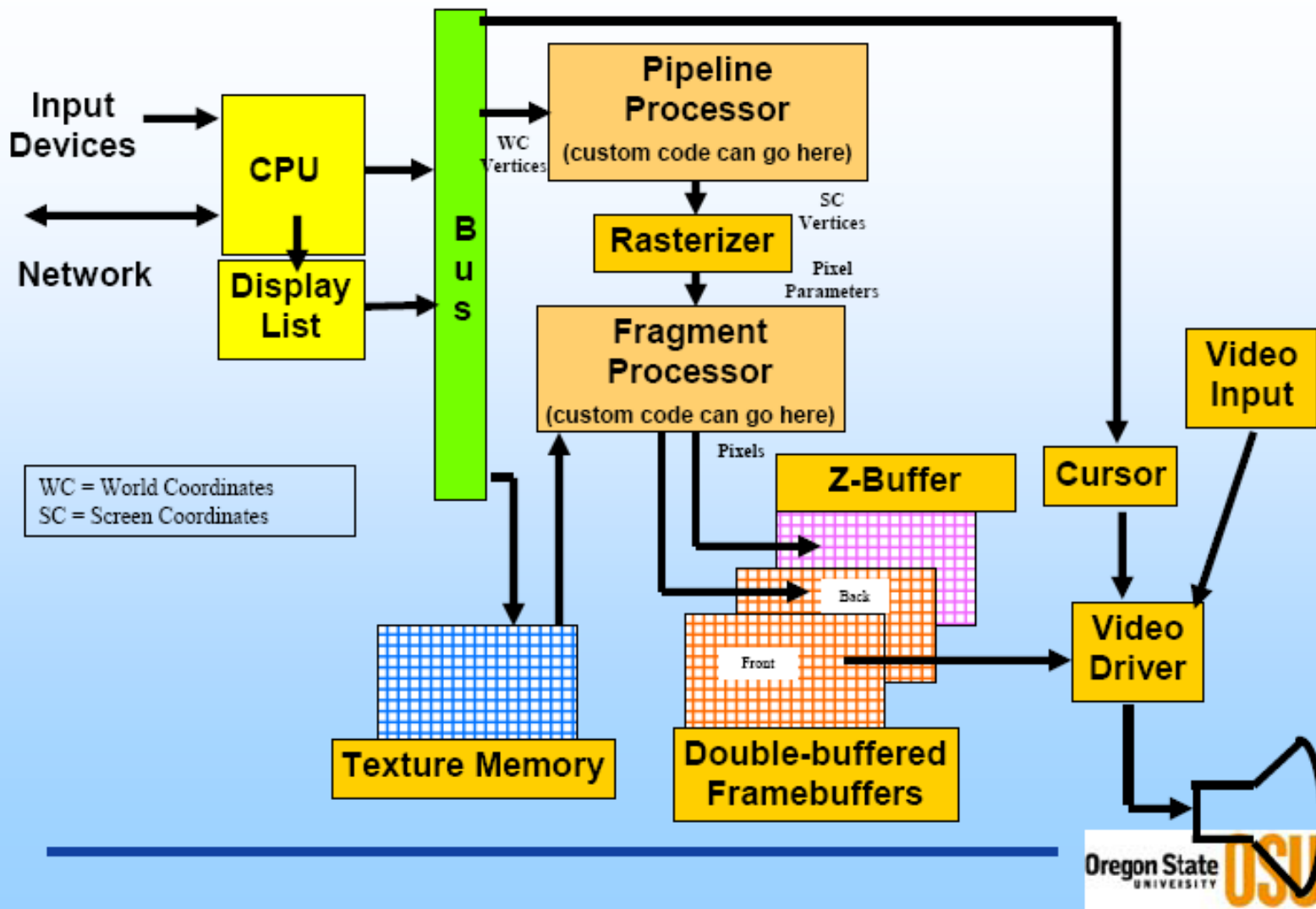
- controllo di flusso molto semplice (control unit ridotta)
- limitata località spaziale (parallelismo a granularità fine)
- alta intensità aritmetica che nasconde le latenze di load/store (cache ridotta)



Generico Sistema Grafico



Generico Sistema Grafico





Pipeline Grafica

Le GPU implementano in hardware la **pipeline grafica** per velocizzare la rappresentazione grafica 3D (accelerazione grafica 3D).

La **pipeline grafica** è la sequenza delle operazioni per ottenere una immagine 2D a partire da una scena 3D; si assume che il mondo 3D sia fatto di **punti**, **segmenti** e **triangoli**.





Pipeline Grafica

Le **GPU** si sono evolute da un'implementazione hardware della **pipeline grafica** ad un substrato computazionale programmabile in grado di supportarla.

Le unità per trasformare i vertici e i pixel sono state incluse in una **griglia di processori**, o **shader**, in grado di eseguire queste attività. Questa evoluzione ha avuto luogo nel tempo sostituendo gradualmente i singoli stadi della pipeline grafica con unità programmabili.

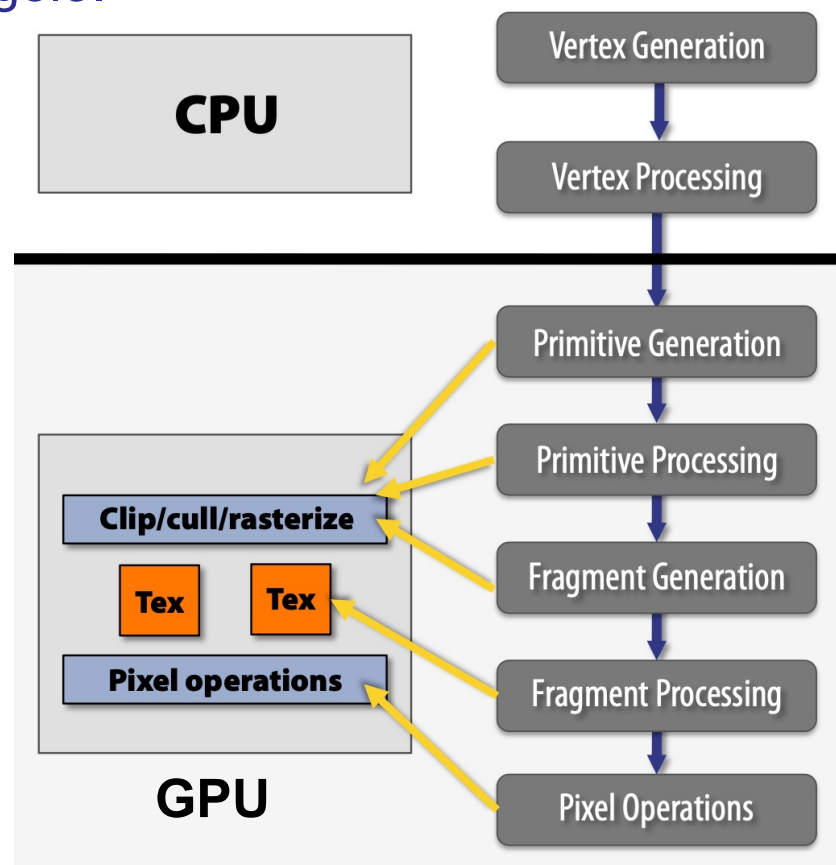
Ad oggi si sono succedute varie generazioni di **GPU**. Oggi siamo a sistemi grafici per il '**rendering offline**' in realtime (**Nvidia RTX serie 40 Ada Lovelace** anno 2022).

Schede Grafiche

GPU con pipeline fissa

I generazione (- 1998) GPU a chip singolo.

- TNT (NVIDIA), RAGE (ATI), Voodoo3 (3dfx);
- trasformazione vertici su CPU, elaborazione pixel su GPU, insieme limitato di operazioni matematiche su pixel;

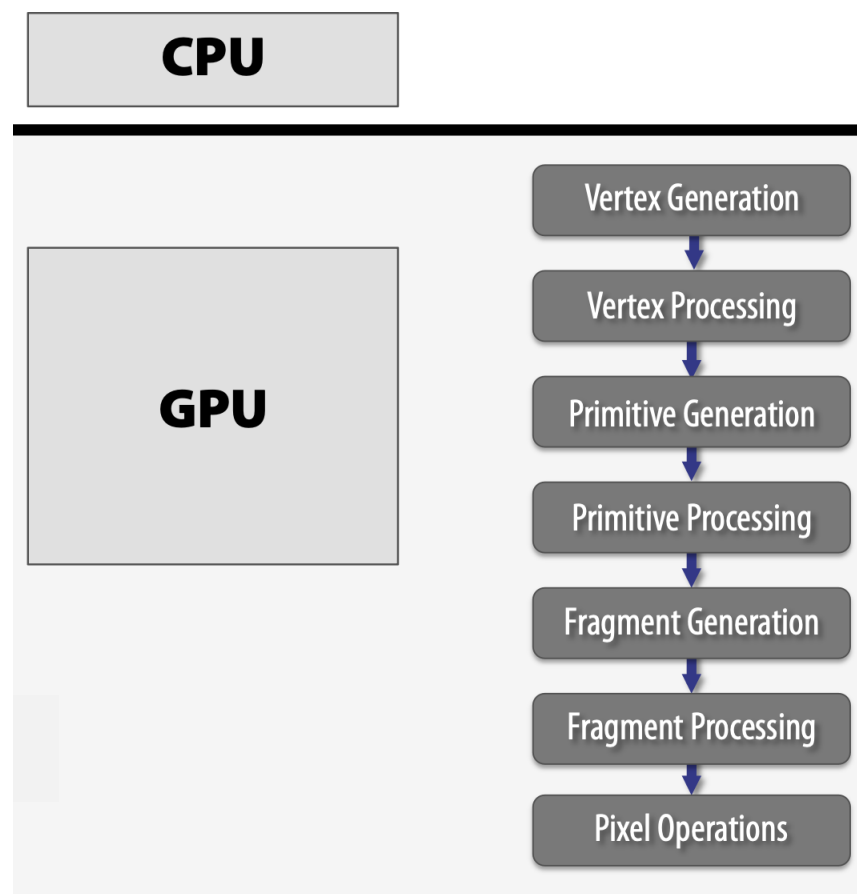


Schede Grafiche

GPU con pipeline fissa

II generazione (1999 - 2000)

- GeForce2 (NVIDIA), Savage3D (S3), Radeon 7500 (ATI), ;
- la trasformazione e illuminazione vertici vengono eseguite anche in hardware (pipeline grafica fissa);





Schede Grafiche

GPU con pipeline programmabile

III generazione (2001)

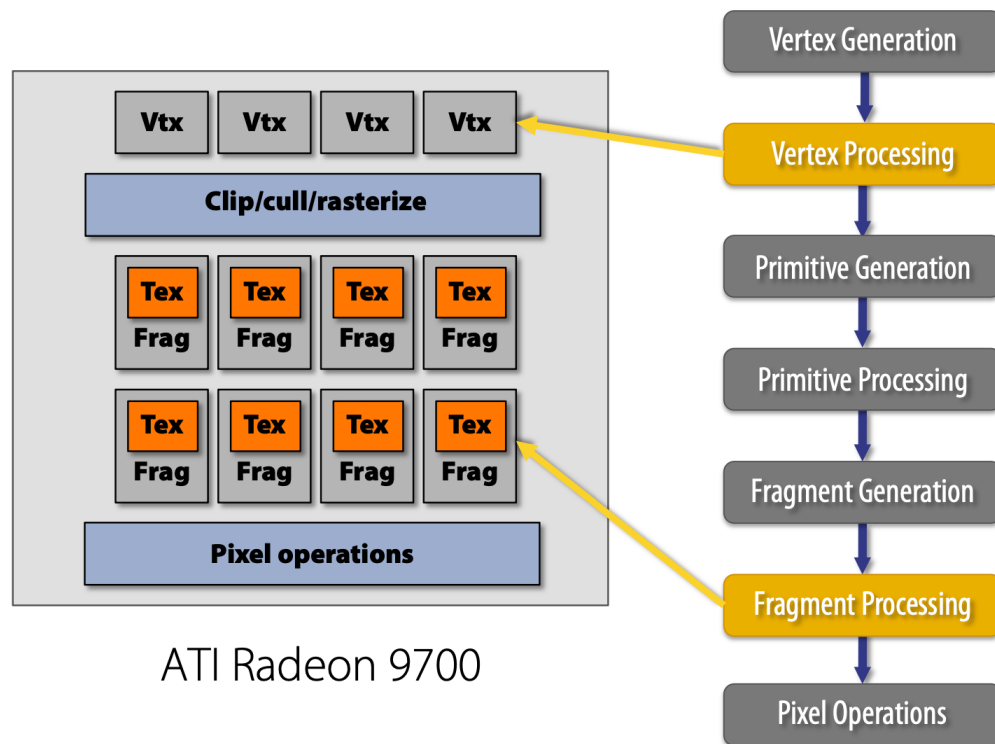
- GeForce3, GeForce4 (NVIDIA), Radeon 8500 (ATI);
- vertex shader programmabili, la scheda grafica consente ai programmatori di caricare assembly program per controllare trasformazione e illuminazione dei vertici; nessuna programmabilità dei pixel;

Schede Grafiche

GPU con pipeline programmabile

IV generazione (2002-2006)

- GeForce FX (NVIDIA), Radeon 9700 (ATI), Quadro4 XGL (NVIDIA), NVIDIA rilascia Cg;
- programmabilità per vertice e per pixel. Aumento dell'uso di effetti di illuminazione. Utilizzo di 32 bit in virgola mobile;

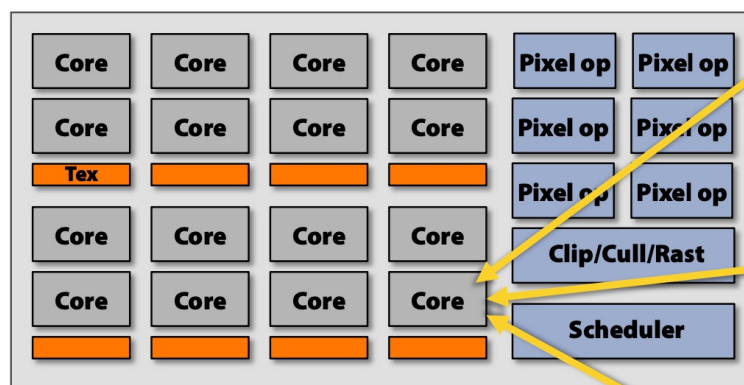


Schede Grafiche

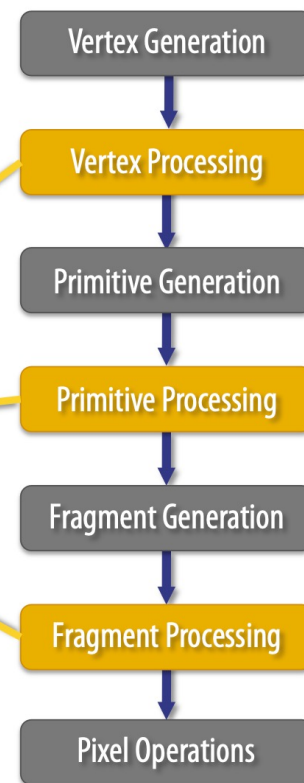
GPU con pipeline programmabile

V generazione (2007 - 2017) GPGPU (GPU General Purpose)

- XBOX360, GeForce 8800 (128 proc.), 32 bit in virgola mobile in tutta la pipeline, GeForce 6+, INTEL (Larrabee), NVIDIA GeForce GTX285, AMD Radeon HD 4890, NVIDIA Tesla/G80;
- shader unificati, introdotti nelle console dei videogiochi.



NVIDIA GeForce 8800
("unified shading" GPU)





Schede Grafiche

GPU con pipeline programmabile

VI generazione (2018 -) Era dei Compute Shader e Ray Tracing. Introdotta con la serie RTX 20 (architettura Turing), aggiunge core specializzati per il ray tracing in tempo reale e Tensor Core per l'IA (Deep Learning Super Sampling - DLSS). Gli shader diventano anche "mesh shader".

Era dell'Intelligenza Artificiale (Presente/Futuro): si sta passando a un utilizzo intensivo dei Tensor Core e dell'IA non solo per il rendering, ma per la generazione intera di frame (DLSS 3/4) e l'illuminazione basata su reti neurali.

Confronto potenza delle GPU



ATI Radeon HD 4850 X2, 1.2 TFLOPS (2008)

ATI Radeon HD 4870 X2, 2.4 TFLOPS (2008)



IBM's ASCI Red, 1.338 TFLOPS
Computer più veloce al mondo nel 1997

NVIDIA GeForce RTX 5090 (Top di gamma 2025): Offre circa **104 TFLOPS** in precisione singola (FP32). Le sue capacità di AI (Tensor Core) superano gli 800 TFLOPS FP16/FP8.



Nvidia e ATI Radeon (ad oggi)

<https://it.wikipedia.org/wiki/NVIDIA>

<https://it.wikipedia.org/wiki/Radeon>

Dispositivi Grafici per l'interattività

- Tastiera
- Penna ottica
- Tavoletta grafica
- Joystick
- TrackBall
- TrackPad
- Mouse
- Touch Screen
- Scanner 3D Touch Probe
- Game Pad
- Leap



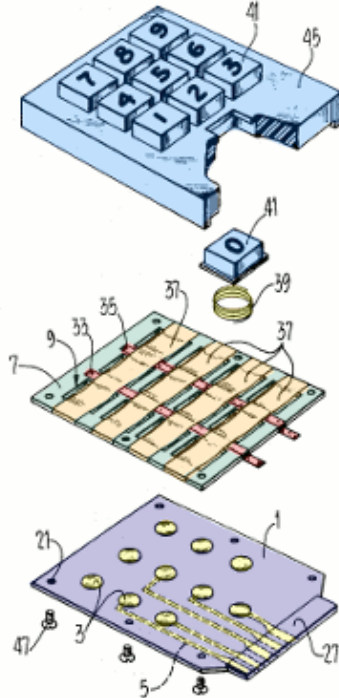
The Leap Motion Controller

Tecnologie dei Dispositivi Grafici

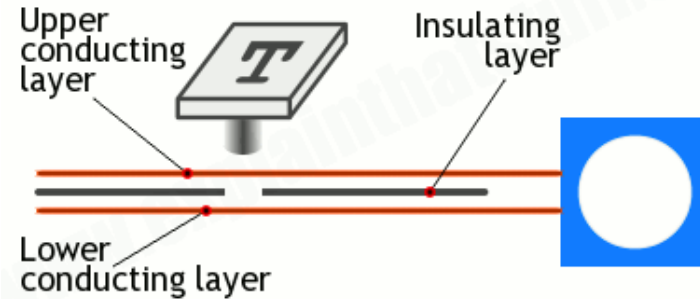
Tastiera

United States Patent

[72] Inventor **James Martin Comstock**
Livermore, Calif.



Courtesy US Patent & Trademark Office
www.explainthatstuff.com



James Martin Comstock, The Singer Company,
depositato nel 1969 e concesso nel 1971.

Tecnologie dei Dispositivi Grafici

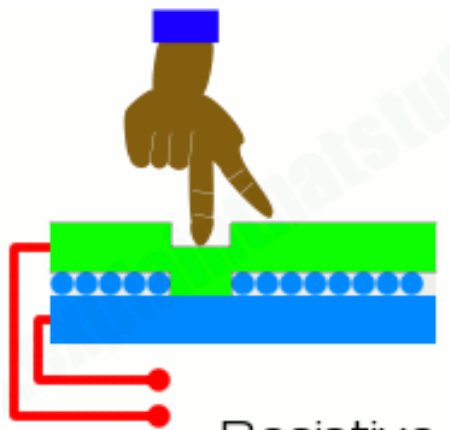
Mouse: la tecnologia attuale è quella ottica

1. Il LED che trasmette la luce verso la superficie d'appoggio;
2. la fotocellula che intercetta la luce riflessa dalla superficie;
3. il chip che si occupa della traduzione del segnale;
4. il sensore che intercetta i movimenti dell'eventuale rotella centrale del mouse;
5. lo switch che intercetta la pressione dell'eventuale rotella centrale del mouse;
6. lo switch che intercetta la pressione del tasto sinistro del mouse;
7. lo switch che intercetta la pressione del tasto destro del mouse;
8. il cavo di collegamento mouse-computer.

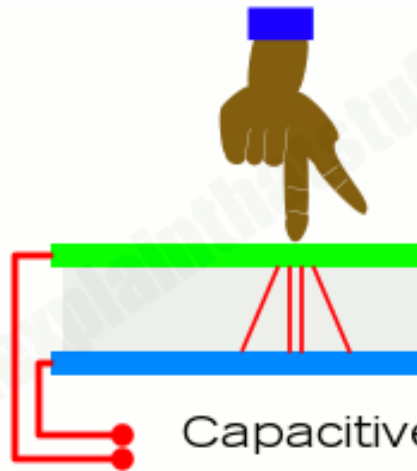


Tecnologie dei Dispositivi Grafici

Touch Screen: ci sono diverse tecnologie, le principali



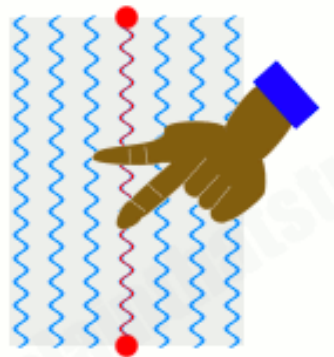
Resistive



Capacitive



Infrared



Surface acoustic wave



Near-field imaging



Il problema dell'interattività

(interattività grafica asincrona)

1. Se ci sono più dispositivi di input (per es. tastiera e mouse) , il programma non è in grado di prevedere quale verrà usato per primo;
2. Anche se ci fosse un solo dispositivo, non è possibile prevedere quando verrà usato.

Presi insieme, questi due problemi escludono l'uso di una classica istruzione bloccante come `"scanf( )"` per leggere i dati in input.

Dobbiamo capire cosa succede quando, mandato in esecuzione un codice che prevede un input da più dispositivi, l'utente ne utilizza uno, per esempio muove il mouse.



Il problema dell'interattività (interattività grafica asincrona)

1. L'utente muove fisicamente il mouse;
2. Il dispositivo (mouse) rileva uno spostamento (delta x, delta y);
3. Il dispositivo invia queste informazioni al computer;

da evento fisico a segnale digitale

4. Il Sistema Operativo riceve l'evento del mouse;
5. Traduce il segnale in un evento comprensibile;
6. Decide quale applicazione deve riceverlo;

il Sistema Operativo fa da intermediario



Il problema dell'interattività (interattività grafica asincrona)

6. L'evento viene inserito in una coda degli eventi;
7. Ogni applicazione ha la sua coda;
8. Gli eventi sono gestiti uno alla volta;

nessuna perdita di eventi

9. L'applicazione esegue un ciclo chiamato event loop; il ciclo:
 - Attende un evento;
 - Lo legge dalla coda;
 - Lo gestisce chiamando una funzione detta event handler;
10. Se non ci sono eventi, l'applicazione resta in attesa.



Coda degli Eventi

Per trasmettere i dati di input al programma utente in maniera corretta, si utilizza una **coda degli eventi**; questa è costituita da una lista di azioni dell'utente o eventi.

Ogni **software grafico interattivo** possiede delle funzioni per accedere alla coda degli eventi;

Esempi:

NextEvent(e) È una funzione bloccante; attende che si verifichi un evento, quindi restituisce le informazioni sull'evento nella struttura **e**

QueueEvent() È una funzione NON bloccante; ritorna il numero di eventi presenti nella coda



Programmazione Event Driven

La **programmazione ad eventi** è un paradigma di programmazione. Nei programmi di questo tipo il flusso è determinato dal verificarsi di eventi esterni.

Il codice è costituito da un **event loop** all'interno del quale le istruzioni controllano la presenza di informazioni da gestire (per esempio il motion del mouse o la pressione di un tasto della tastiera), e quindi nell'eseguire quella parte di codice scritto appositamente per gestire l'evento in questione.

I programmi **event-driven** fanno uso di funzioni chiamate **gestori degli eventi** (**event handlers**), eseguite in risposta agli eventi esterni, e di un **dispatcher**, che effettua materialmente la chiamata, a seguito dell'accesso alla **coda degli eventi** che contiene l'elenco degli eventi già verificatisi, ma non ancora "processati".

I programmi con interfaccia grafica sono realizzati secondo il **paradigma event-driven**.



Programmazione Event Driven

```
main( )  
{  
  int done;  
  ....  
  done=TRUE  
  while (done)  
  {  
    NextEvent(e )  
    switch (e.type)  
    {  
      .....  
      .....  
    }  
  }  
}
```

Esempio 1

```
main( )  
{  
  int done;  
  ....  
  done=TRUE  
  while (done)  
  {  
    if (QueueEvent( ) >0)  
    {  
      NextEvent(e)  
      switch (e.type)  
      {  
        .....  
        .....  
      }  
    }  
  }  
}
```

Esempio 2

Esempi di **event loop** nel programma applicativo



Esempio: dragging di un'immagine

Problema: dal `ButtonPress` del mouse al `ButtonRelease` si vuole spostare un'immagine seguendo la posizione (coord. (x,y)) del mouse.

Le coordinate del mouse entrano nella coda degli eventi come eventi di **Motion**

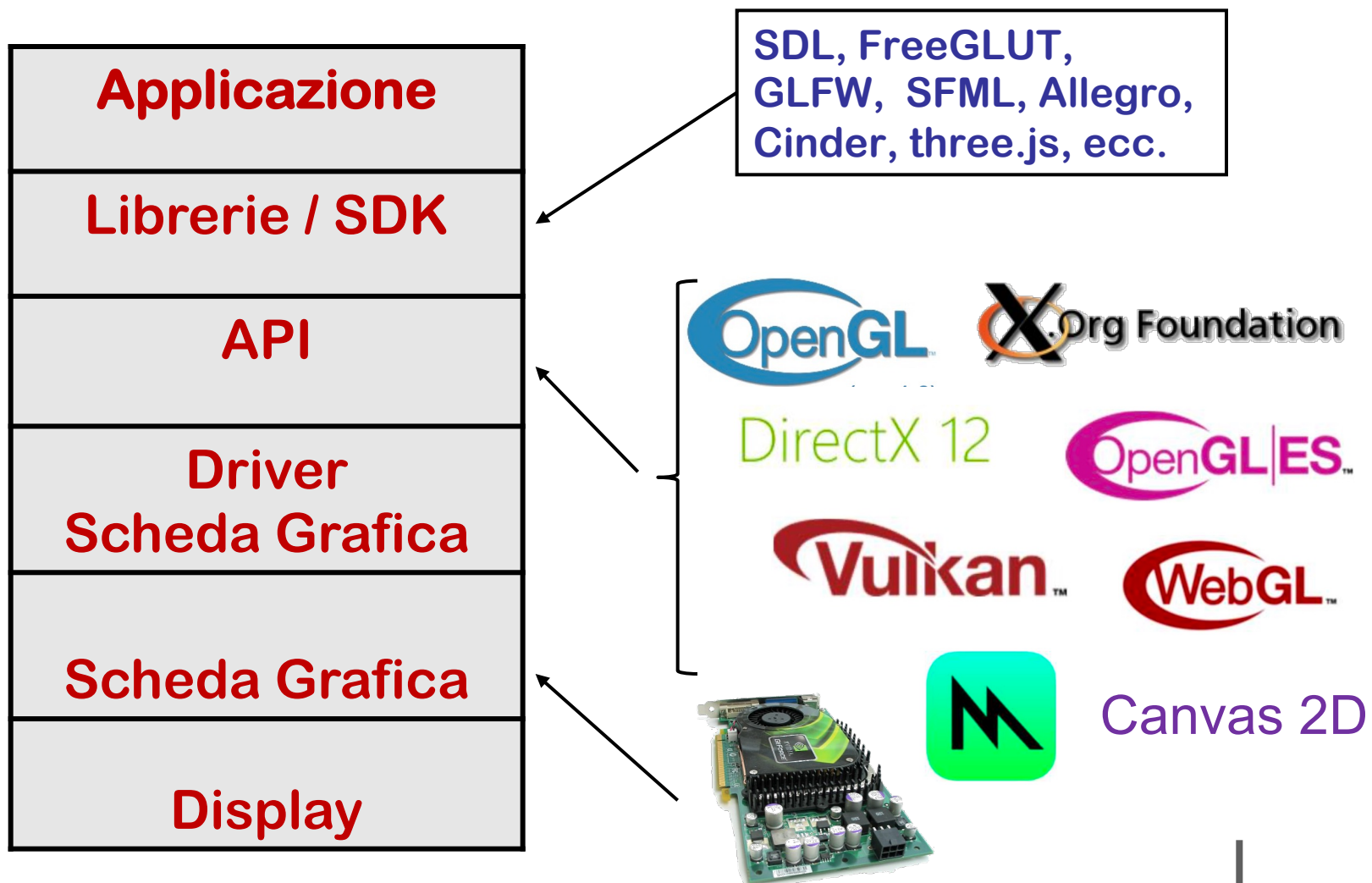
Se l'immagine non è banale e il suo ridisegno comporta un certo tempo, si potrebbe rilevare una incongruenza tra la posizione del mouse e l'immagine; si dice che non c'è **feedback**

Questo è dovuto al fatto che il sistema non riesce a processare gli eventi man mano che vengono generati e quando l'applicazione li processa sono diventati vecchi;

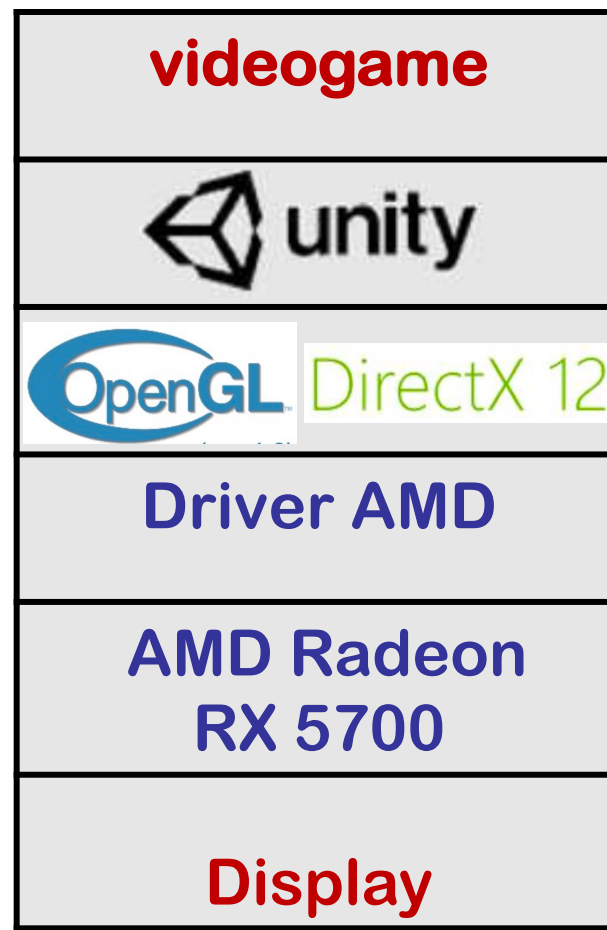
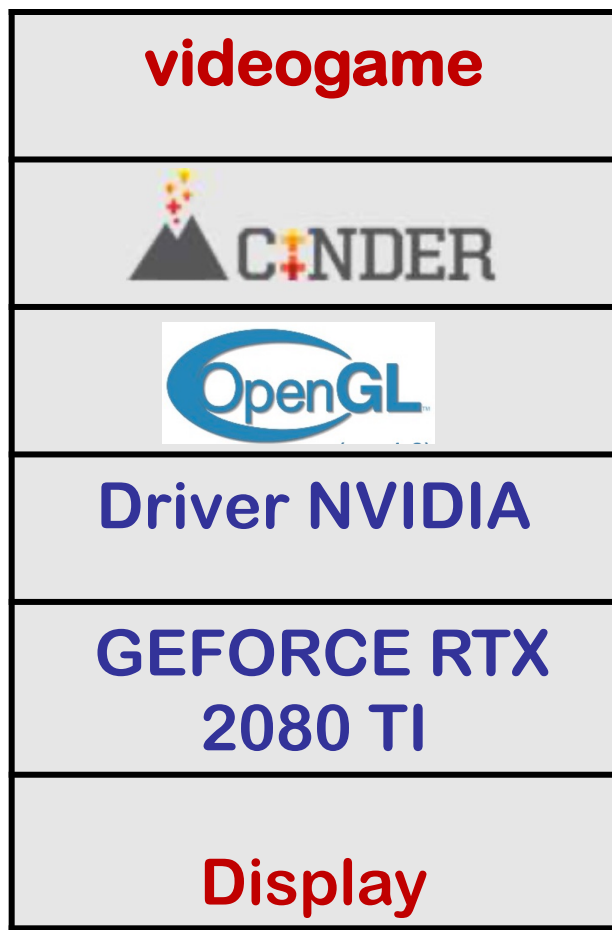
in questo caso sarebbe meglio che la coda non contenesse mai più di un evento alla volta.

PermitEvent(t) Consente di aggiungere un evento di tipo **t** alla coda degli eventi.

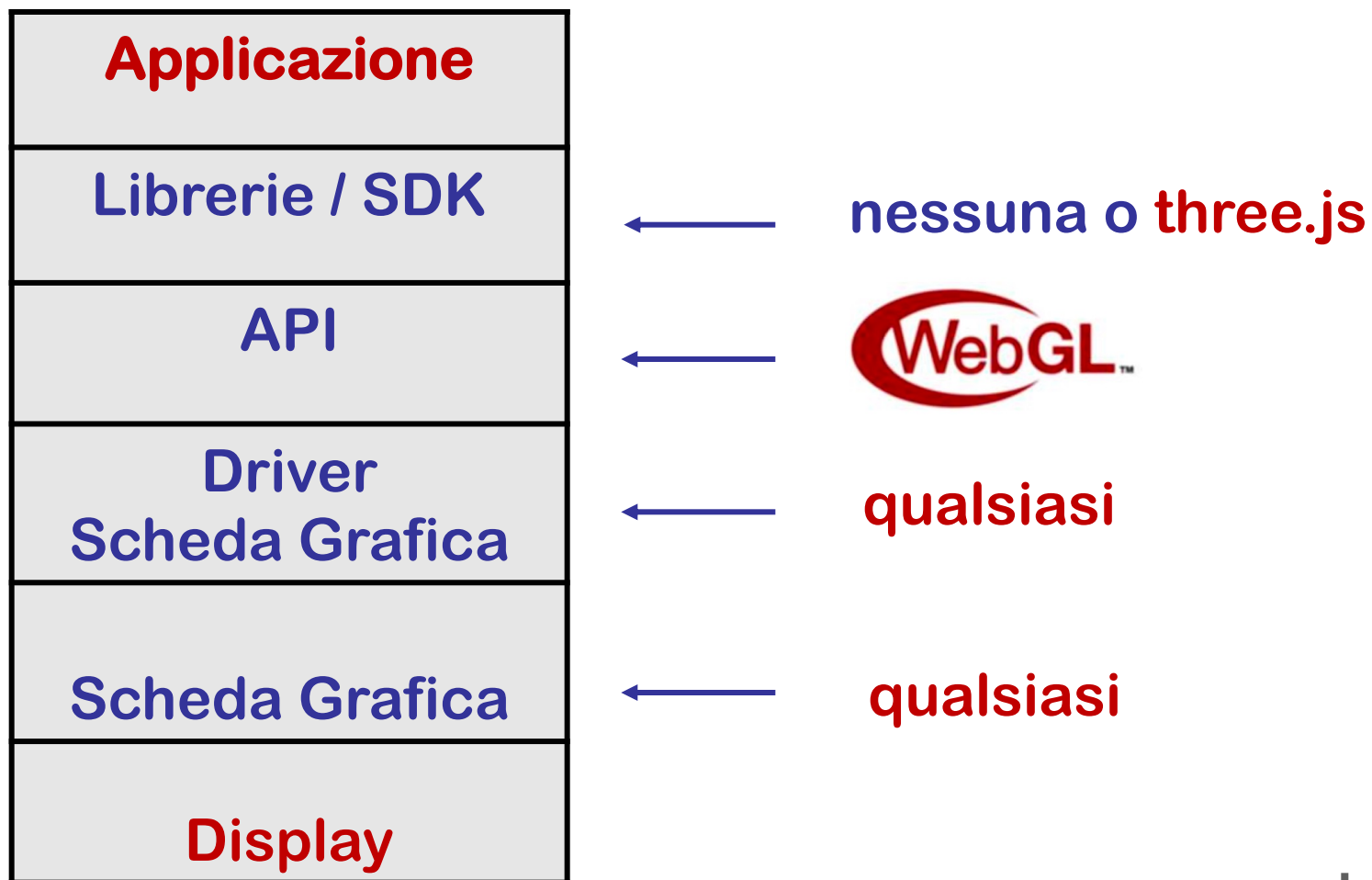
Ambiente Grafico



Esempi



I nostri progetti





ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Giulio Casciola

Dip. di Matematica

giulio.casciola@unibo.it